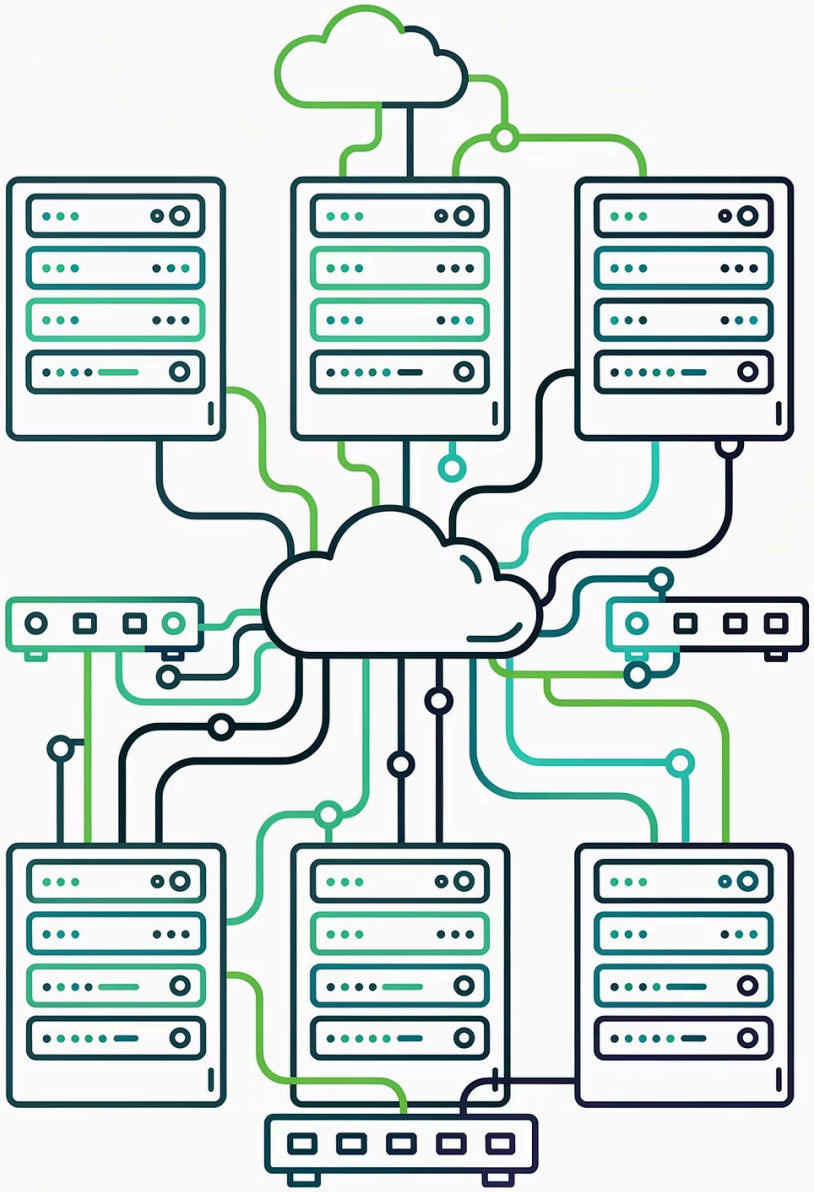
# Module 6: Cost Optimization

## Amazon Bedrock — Research Findings · February 2026

Cost optimization for Amazon Bedrock spans model pricing, service tiers, batch vs. online inference, caching, and architectural choices across the full AI application lifecycle. This module provides a structured framework to help cloud architects, ML engineers, and technical product managers make informed decisions that balance performance, quality, and spend.

MODULE 6 | AMAZON BEDROCK | COST STRATEGY



# On-Demand Pricing: Service Tiers

Amazon Bedrock offers three on-demand service tiers that let you explicitly trade off price, availability, and performance to match each workload's requirements.

## ⚡ Flex Tier

**~50% below Standard list prices.** Best-effort, no strong SLA — highest throttling risk among tiers.

- Development, testing, and model evaluations
- Non-urgent batch and async workloads
- Agentic workflows that tolerate variable latency

## ⚖️ Standard Tier

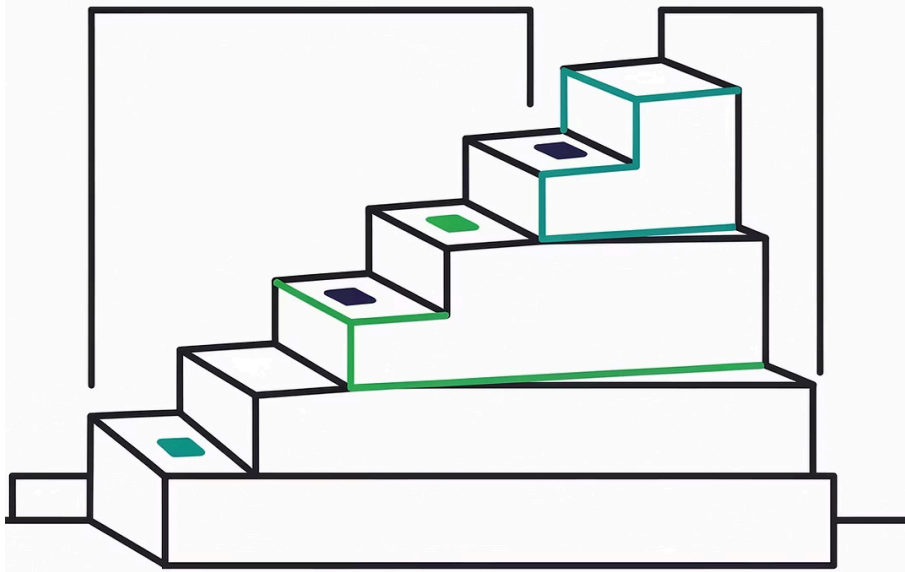
**Baseline rates** on the Bedrock pricing page. Balanced performance and throughput guarantees.

- General production applications
- Content generation, analysis, and RAG
- Consistent workloads with typical SLAs

## 🚀 Priority Tier

**~75% above Standard.** Preferential processing, higher capacity, and up to ~25% better latency/OTPS.

- Mission-critical, customer-facing apps
- Real-time translation and interactive UX
- Latency-sensitive paths where cost is secondary



# Reserved / Provisioned Capacity

## How It Works

Reserve capacity for specific models in advance for predictable pricing and guaranteed throughput. Commitment options include **1-month** and **3-month** (or longer) terms — longer commitments yield better effective rates.

Pricing is fixed for a configured throughput (e.g., tokens-per-minute or model units) for a given model, tier, and Region.

## Break-Even Thinking

Reserved capacity often becomes cost-effective when you have **sustained, predictable usage**. A common internal heuristic: when average utilization is a substantial portion of peak — often cited as **>~40–50% vs. on-demand** — reserved options can start to pay off.

> This is not an AWS-documented threshold. Always run your own math per model and Region before committing.

## Best For

- Consistent, high-volume production workloads
- Long-running apps with predictable traffic
- Teams where budget predictability is a hard requirement

# Batch Processing: 50% Cost Savings

## Key Characteristics

### Cost Savings

~50% off Standard tier on-demand pricing for eligible foundation models in most Regions.

### Processing Window

Jobs processed asynchronously; plan for a **24-hour completion window**, though many finish same-day.

### Job Limits

Records per job, file sizes, and concurrent job limits vary by model and Region. Always consult the batch inference docs for current quotas.

## Best Use Cases

- Training data generation at scale
- Content moderation backfills
- Bulk summarization and report generation
- Data enrichment and historical analysis where latency is not critical

Batch inference is the single easiest cost lever for offline workloads — adopt it before exploring more complex optimizations.

# Prompt Caching

Prompt caching reduces cost *and* latency when a static prompt prefix is reused across many calls — one of the highest-impact, lowest-effort optimizations available on Bedrock.

## How It Works

Bedrock identifies and caches repeated **prompt prefixes** such as system prompts, long instructions, and shared context. On a cache hit, you pay only for **new input tokens (suffix) and output tokens** — cached prefix tokens are billed at heavily discounted or zero rates depending on the model's caching pricing. Cache management, storage, and expiry are handled automatically by Bedrock.

## Supported Models

Initially available on select **Anthropic Claude** models; now extended to **select Amazon Nova** models and other families. Coverage is actively expanding — always verify per-model documentation.

## Savings Potential

~90%

Savings on Cached Portion

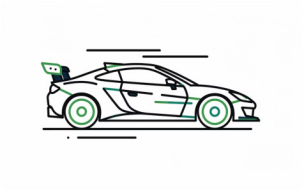
For supported models when long system prompts are reused across many calls.

## Best For

- Apps with stable system prompts or long shared instructions
- RAG systems with fixed instructions and repeated context structures
- Chatbots using a shared policy or prompt across many users

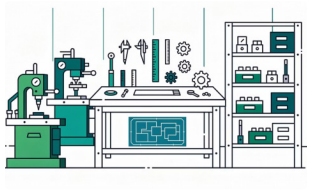
# Model Selection and Right-Sizing

Choosing the right model class is the **highest-leverage cost decision** you make. Always start with the smallest model that meets your quality threshold, then scale up only where it delivers clear business value.



## Light / Small Models

Compact Mistral, smaller Llama 3 family variants, and similar. **Fastest and cheapest** — ideal for classification, routing, basic Q&A, and high-traffic endpoints where quality demands are modest.



## General / Mid-Range Models

Mid-sized Llama 3/3.1/3.2/3.3, mid-sized Nova models, Claude Sonnet. **Best balance** of quality, cost, and latency. Suitable for the majority of production workloads.

## Flagship / Frontier Models

Claude Opus, larger Nova models, largest Llama variants. **Highest reasoning quality** at significantly higher per-token prices and latency. Reserve for tasks where accuracy directly drives business value.

Indicative pricing example (subject to change, region-specific): Claude Sonnet ≈ $3/1M input tokens, $15/1M output tokens. Open-source models are often significantly cheaper per million tokens than proprietary frontier models. Always consult the Bedrock pricing page for current rates.

# Model Distillation

Bedrock's model distillation workflow trains a **smaller "student" model** to mimic a high-quality "teacher" model — dramatically lowering serving cost and latency with minimal accuracy loss.

## Measured Benefits (AWS Benchmarks)

**5×**

### Faster Inference

500% improvement on many workloads vs. the teacher model.

**75%**

### Cost Reduction

Per-inference savings when serving the distilled student model.

**<2%**

### Accuracy Loss

On RAG-style tasks in internal tests — often negligible in practice.

## The Distillation Process

01

### Select a Teacher Model

Choose a high-quality model such as Claude Opus or a large Nova variant.

02

### Generate Training Data

Produce prompt → teacher-response pairs on your specific domain and use cases.

03

### Run Bedrock Distillation

Use Bedrock's built-in distillation workflow to train the student model.

04

### Deploy and Monitor

Serve the student in production; optionally fall back to the teacher for edge cases.

**Best for:** RAG apps, function-calling agents, and high-scale production workloads where serving cost and latency matter.

# Intelligent Prompt Routing

Intelligent Prompt Routing automatically directs each prompt to the most appropriate model within a supported family — optimizing cost and quality *without changing application code*.

## How It Works

A router model or policy classifies each incoming prompt by complexity and use case. **Simple prompts** are sent to smaller, cheaper models; **complex prompts** are escalated to larger, higher-quality models. Routing is typically transparent to application code — you call a family endpoint or routing API and the service handles the rest.

## Key Benefits

- Cost savings without significant quality degradation
- Improved p95/p99 latency for simpler queries
- Reduced manual orchestration complexity in application code

## Example Routing Pattern

1. **Simple FAQs & Classification**
   → Small Llama or open-source model
2. **Complex Multi-Step Reasoning**
   → Claude Sonnet or Nova Pro
3. **Rare Expert / High-Stakes Tasks**
   → Claude Opus or largest Nova model

# Prompt Engineering for Cost Efficiency

## Optimize for Clarity & Conciseness

Shorter, clearer prompts reduce token usage and often *improve* model behavior simultaneously. In practice, many real prompts can be shortened by **30–50% in tokens** while fully retaining intent.

**Techniques:**

- Remove filler words and redundancy
- Use direct language and explicit instructions
- Structure prompts with concise sections: role, context, task, constraints

❌ "I would like you to please help me understand what the weather is like today in New York City if you don't mind."
✅ "What's the weather in New York City today?"

## Use Prompt Caching Effectively

Structure prompts so Bedrock's native caching delivers maximum savings on supported models.

**Strategy:**

- Put **static content first**: system prompt, policies, domain context
- Keep user-specific and dynamic content in the **suffix**
- Rely on Bedrock caching to avoid re-paying for the static prefix

**Pattern:**
system_prompt = """ You are a helpful customer service agent. Company policies: [long policy document] """
user_query = "What is your return policy on electronics?"

## Few-Shot vs. Zero-Shot

Every example you add costs tokens. Be deliberate about when examples actually improve outcomes enough to justify the cost.

### Zero-Shot

No examples. **Lowest token usage.** Always the starting point.

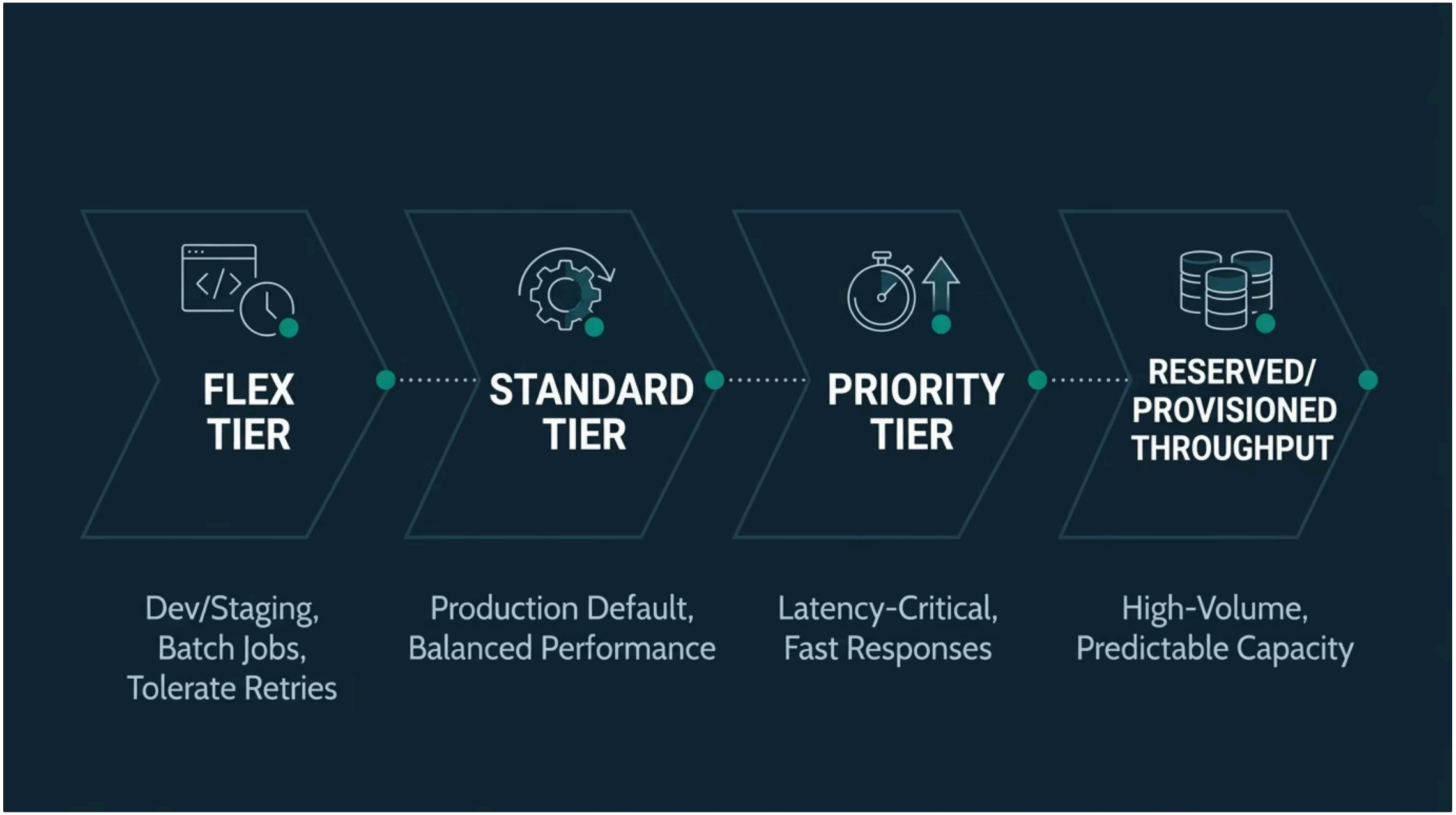
### Few-Shot

Add examples only where they **significantly improve quality.** Keep examples minimal.

Consider distilling few-shot behavior into a student model to eliminate repeated example tokens at scale.

# Service Tier Selection Framework

Use this decision framework to assign the right tier to each workload or traffic path — avoid over-provisioning with Priority when Standard or Flex will do.




Treat ">40% of on-demand cost" as your team's own rule-of-thumb for exploring reserved options — not an AWS guarantee. Run your own break-even math per model, Region, and commitment term.

# Agent Design Optimization

## Build Small, Focused Agents

One giant agent with 20+ tools, broad instructions, and long context leads to high token consumption and unpredictable behavior. Instead, decompose into **multiple specialized agents** with clear delegation.

### ✅ Best Practice

Specialized agents per domain (billing, orders, tech support) coordinated by a supervisor agent or routing logic. Short, domain-bounded prompts and efficient tool selection.

### ❌ Anti-Pattern

Single monolithic agent with 20+ tools, broad instructions, and unbounded context windows driving excessive token consumption.

## Optimize Tool Descriptions

Clear, precise tool descriptions reduce unnecessary tool calls and costly retries. Every ambiguous description is a hidden tax on your token budget.

**Include in every tool description:**

- **What** the tool does and **when** to use it
- Expected **inputs** and **outputs**
- A simple inline example if the schema allows

❌ "Gets order info"
✅ "Retrieves order status, shipping info, and tracking number for a given order ID. Use when the customer asks about order status or shipment details."

# Knowledge Base Optimization

## Manage Data and Indexing Frequency

Embedding generation is typically a one-time cost per chunk — but re-embedding a large corpus unnecessarily can become a significant recurring expense. Vector storage costs also grow with chunk count and dimensionality.

- Use **incremental sync** to re-embed only changed documents
- Schedule large syncs during **off-peak hours**
- Avoid full-corpus re-embedding unless content changes broadly

## Optimize Retrieval Settings

Every retrieval parameter is a cost lever. Higher top-k improves recall but increases token consumption linearly — tune based on evaluation, not intuition.

### Top-k / Chunk Count

Start with **5–10 chunks** and tune based on answer-quality evaluation. Higher k = higher token + retrieval cost.

### Reranking

Adds an extra model call but improves precision. Use when precision matters more than minimal cost; skip for highly cost-sensitive workloads.

# Caching Strategies

## Application-Level Caching

Cache frequently requested queries and responses at the application layer to eliminate redundant model invocations entirely. Particularly impactful for stable FAQ-style content and expensive RAG + generation flows.

**What to cache:**

- Frequent queries and their responses
- Static responses (e.g., common FAQs)
- Expensive RAG + generation flows with stable answers

**Implementation sketch:**

```
cached = cache.get(query_hash)
if cached: return cached
response = bedrock_runtime.invoke_model(...)
cache.set(query_hash, response, ttl=3600)
return response
```

Use Redis/ElastiCache for low-latency ephemeral caching; DynamoDB or S3 for persistent caching.

## Bedrock-Native Prompt Caching

Bedrock's built-in prompt caching works automatically for supported models once enabled — it's transparent to application logic. You simply structure prompts with reusable static prefixes and Bedrock handles the rest.

### Structure First

Place all static content (system prompt, policies) at the start of the prompt prefix.

### Combine Both Layers

App-level caching + Bedrock prompt caching together deliver maximum cost savings for high-traffic applications.

# Batch Processing Strategy

## The Core Value Proposition

Batch inference delivers approximately **50% cost savings** vs. Standard on-demand pricing for eligible models — with no infrastructure management overhead. Bedrock handles capacity utilization and job scheduling for you.

---

**When to use it:**

- Large-scale, non-interactive processing
- Historical data moderation and enrichment
- Bulk summarization and report generation
- Training data generation pipelines

## Trade-Offs to Understand

### ✅ Batch

~50% cost savings. Scales to large job volumes. No worker fleet required. Predictable asynchronous processing within a 24-hour planning window.

### ⚡ Real-Time

Low latency and immediate responses. Required for interactive, user-facing workloads. Higher per-token cost at Standard or Priority tier rates.

Batch inference is often the **fastest path to 50% savings** for teams with offline processing workloads. Evaluate before investing in more complex optimizations.

# Monitoring and Cost Tracking

You cannot optimize what you do not measure. Instrument every Bedrock workload from day one — cost surprises at scale are painful and avoidable.

## CloudWatch Metrics to Track

- **Token usage** per model, per tier, and per application
- **Request counts** and latency statistics (p50, p95, p99)
- **Error and throttle rates** — retries drive hidden token costs
- **Anomalous token consumption** — may indicate bugs or abuse

Set CloudWatch alarms for unexpected usage spikes, latency/error SLO breaches, and anomalous consumption patterns.

## Cost Allocation Best Practices

### Tagging Strategy

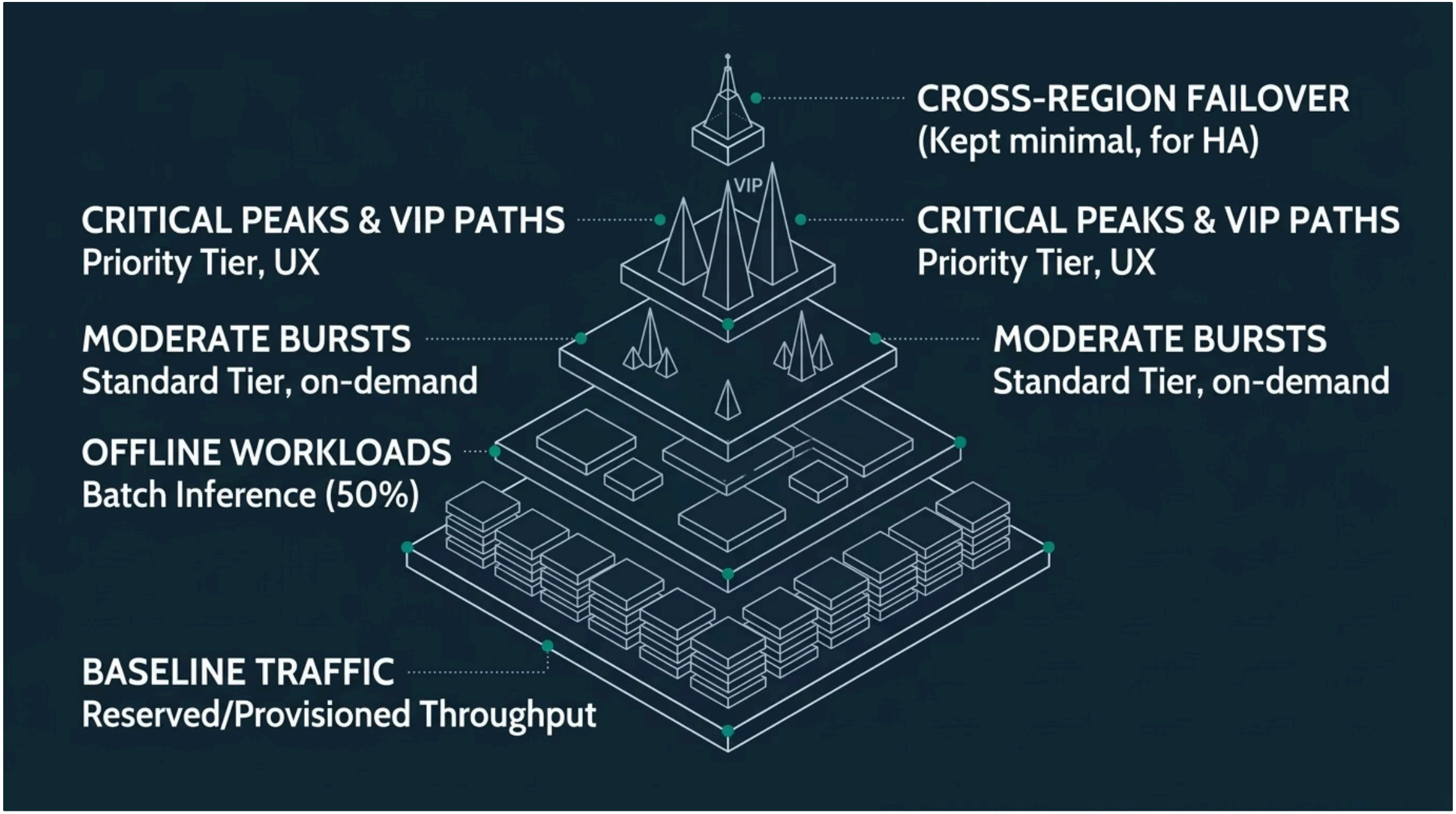
Tag every invocation by **environment** (dev/stage/prod), **team**, **project**, and **tenant** to enable per-team cost allocation in Cost Explorer.

### Inference Profiles

Log an **application identifier** in each call to build per-app cost views. Combine CloudWatch, Cost Explorer, and logs for granular dashboards.

# Combining Approaches: The Full Cost Strategy

No single lever delivers optimal results. A robust cost strategy layers multiple techniques across all traffic patterns simultaneously.




Layer reserved capacity at the base for cost predictability, add on-demand tiers for burst handling, and reserve Priority tier for the smallest slice of truly latency-sensitive traffic. Batch inference handles the long tail of offline workloads at half price.

# Cost Monitoring Best Practices

1

## Set Up Cost Dashboards

Track daily token usage by model and tier, cost per model and per application, and trend lines vs. budgets. Use Cost Explorer with tag-based filters for team-level visibility.

2

## Implement Budget Alerts

Configure AWS Budgets and/or Cost Anomaly Detection with alerts at **50%, 80%, and 100%** of your monthly budget. Pair with CloudWatch/SNS for real-time notification to on-call engineers.

3

## Conduct Regular Cost Reviews

Weekly or monthly reviews of top cost drivers. Systematically evaluate model choices, tier assignments, caching effectiveness, and batch conversion opportunities before committing to changes.

4

## Run A/B Tests for Optimization

Test different models, prompt variants, caching levels, and tier assignments side-by-side. Measure **cost per request vs. quality vs. latency** to make data-driven decisions — not assumptions.

# Real-World Cost Optimization Patterns

These illustrative examples show how layering multiple strategies produces compounding savings. Treat them as patterns to adapt, not official case studies.

## Customer Service Chatbot

**Before:** Frontier model (Claude Opus) for all queries, Priority tier, no caching — ~$5,000/month.
**After:** Mid-tier model (Claude Sonnet/smaller Nova) for most queries; Intelligent Prompt Routing escalates complex cases to Opus; Standard tier for most traffic; prompt + app-level caching for FAQs.
**Result: ~$1,200/month (~75–80% savings)**

## Document Processing Pipeline

**Before:** Real-time per-document inference, Standard tier only, no batch — ~$8,000/month.
**After:** Bulk summarization and moderation moved to batch inference (50% rate), jobs scheduled off-peak, smaller classification model for simple docs.
**Result: ~$2,800/month (~65% savings)**

## RAG Application

**Before:** Large model for all queries, 20 chunks retrieved each time, no caching — ~$3,500/month.
**After:** Intelligent Prompt Routing + mid-tier model; prompt caching for shared instructions; top-k reduced from 20 to 8 after evaluation; frequent-query result caching.
**Result: ~$900/month (~70–75% savings)**

# Key Takeaways

Achieving meaningful Bedrock cost savings requires a multi-dimensional, iterative approach — not a single switch to flip.

### Multi-Dimensional Optimization

Combine model selection, service tiers, batch inference, prompt caching, and architecture patterns. No single lever achieves optimal savings alone.

### Measure Everything

Token usage, latency, error rates, and per-application costs are *required inputs*, not nice-to-haves. Instrument from day one.

### Iterate Incrementally

Start simple, instrument, then optimize models, prompts, and tiers based on real data. Avoid premature optimization before you have usage baselines.

### Quality vs. Cost

Use structured experiments and RAG/agent evaluations to find the right balance. The cheapest model that meets your quality threshold is the right model.

### Automate Cost Efficiency

Leverage Intelligent Prompt Routing, model distillation workflows, and batch scheduling to make cost-efficiency systematic rather than manual.